Programmable spaces and their extensional meanings

October 2026

A space stores atoms. A requested agenda interprets some of those atoms under a declared source language and execution policy. Several such languages can share the same store. What may be identified depends on the observation and the consumer: current facts, ordered evidence and future behaviour have different equality relations.

Execution contract. `Core.ProgrammableSpace.Language` separates source admission, scoped initialization, residual states, source events, outcomes and receipts. `Policy` independently restricts source events and retains changing policy state. A session retains the original source occurrence, its snapshot, scope, request and residual. A recorded event retains both stores, both residuals, both policy states and the actual source receipt. Launching an agenda is an explicit action. An atom generated by one language does not execute merely because another language recognizes its spelling.

The operational contract is qualified by `Started`: a space is reachable from an initial stored list with no sessions through actual launches and source steps. This entails admitted original requests and valid event histories. Public record constructors alone establish neither property. A rejected atom may still be stored as inert data. List and array representations implement the exact occurrence store; a membership-only store cannot reconstruct its duplicates.

Authored scoped rewriting instantiates this contract using the existing finite-premise-depth executor and its actual rule histories; the lambda instance interprets intrinsically scoped beta reduction. The symbolic wire codec retains binder indices and metadata. The MM2 adapters reuse the existing rule-scoped source parser, compact-key matching snapshot and physical rows. The native MM2 syntactic profile accepts comma inputs or explicit `I` inputs containing unary `BTM` factors, with unary `+/-` sinks under `O`. The admission theorem characterizes exactly these forms, including empty explicit inputs. The native `take`, `all` and `absent` operations belong to other resource profiles; equality and inequality factors in the wider Lean source language are not native MM2 parser forms. This instance does not assert full upstream MM2 compatibility. Source-step comparisons concern admitted, selected requests. The native protocol can retain an unsupported directive as pending work; rejection by this language's admission relation does not assert a native consumption step.

MM2 has two distinct effect-order profiles. The existing source executor applies each sink across every matched row; the native provider applies all sinks for one row before the next. For matches $(a, b), (b, c)$ and sinks $+x, -y$, the former retains a and the latter retains a, b, in addition to the original edge facts. The row-major language therefore has its own source transition, finalizer, outcomes and resumable comparison. General agreement holds for physical-key support when every pair of instantiated writes either has the same polarity or addresses different keys. This sufficient condition covers positive insertions; it does not equate literal representatives, order, receipts or work. Changing effect order changes the language, whereas a policy selects among transitions already licensed by that language.

The resumable MM2 adapter retains the actual structural matching packet, private ordered rows, granted budget and accumulated poll charge. A paused cursor cannot publish. A completed cursor publishes exactly its profile's atomic source result. Resumption requires the same retained store; a concurrent mutation requires an explicit validated restart or snapshot policy. Formal cursor polls and the native runtime's work counter are distinct accounts. The native agenda takes ownership of its selected directive before matching. Its physical live store can therefore differ during a pause. Reinserting the owned directive recovers logical key support precisely when that key was originally present; actual selection establishes this condition. This proved representation comparison does not verify the native capture/retry protocol.

The concrete mixed-space run pauses MM2 after one private answer, performs authored beta while retaining the exact snapshot and matching packet, then resumes and publishes both answers. Actual shared-space

steps establish this interleaving, with admitted origins, separate policy accounts and a decoded material continuation for the publication step.

Three readings.

Reading	Equality identifies	Evidence retained separately
Current fact support	Exactly the same literal members	Order, duplicates, premise occurrences, policy and work
Retained derivations	Exactly the declared proof structure or receipt fields	Authored rule positions, ordered premises, substitutions and origins
Observed behaviour	Contextual bisimilarity preserving the declared readings	Original spaces remain available before taking this quotient

The atom graph dictionary is constructed for every source atom constructor, including UTF-8 strings, custom grounded fields and ordered expressions. Consequently the fact-support graph has exactly the membership kernel. Literal atom identity is distinct from MM2's compact-key alpha comparison. A second constructed dictionary retains both compact byte keys and abstract atom keys. Its material value identifies exactly physical-key support and transports the effect-order agreement above. Two differently named variables can agree in this reading while remaining distinct in the literal reading. The complete typed derivation reading retains the recursive proof tree; the MM2 receipt projection retains its selected literal instruction, entire ordered source snapshot and all ordered substitution/witness rows. The latter does not claim injectivity on unobserved request fields. Explicit-factor source witnesses are in source order, while the cursor retains a reverse stack; their proved conversion preserves substitutions and physical origins. A reflective self-read distinguishes `I/BTM` from comma syntax, so the original instruction is retained rather than normalized away. Older paired source/runtime examples preserve their respective literal instructions separately; comparisons of facts and receipt fields do not identify differently encoded priority syntax. The conflicting-write instance uses the same integer-valued priority in both sources.

For behaviour, stage n contains the reachable spaces with at most n retained session births and events. Context extension retains the entire space. Every actual action has its actual successor at the next stage, and existential future-child membership reflects exactly the original action relation. World and context-arrow labels remain observable in the contextual material kernel. That kernel must not be renamed as unstaged bisimilarity: same-stage successor availability can distinguish different extents. The existing constructed material recipient, labelled hyperset reading and dependent-family interpretation agree on this declared kernel. Formula interpretation preserves and reflects each supported finitary modal formula; this does not establish unrestricted Hennessy–Milner characterization or predecessor-modal descent from future bisimulation alone.

Persistent inference and policy. Let I index actual ground rule instances, with finite ordered premises B_i and conclusion h_i, and let S_0 be the initial facts. An execution selects an instance or idles; a successful step adds h_i when $B_i \subseteq S_n$ and retains every existing fact. Rule-instance fairness means that every instance enabled at some stage successfully fires at some stage. Recurring a rule template is insufficient: a template may repeatedly serve one substitution and starve another.

For every such fair run,

$$\bigcup_{n<\omega} S_n = \{a \mid a \text{ has a finite derivation from the authored rules and } S_0\}.$$

Thus all fair policies have the same eventual material fact support. A finite prefix equals that closure precisely when it is rule-closed. The successor instance $0, 1, 2, \ldots$ has infinite eventual support and no complete finite prefix. Arbitrarily delayed fair schedules refute a uniform finite-budget equality claim. Destructive updates, provenance and cost require their own policy-sensitive observations.

The native scheduling control starts with `(p a)`, a consumer $p(x) \Rightarrow q(x)$, and a producer of `(p b)`:

Language	Consumer first	Producer first
One-shot `exec`	$p(a), p(b), q(a)$	$p(a), p(b), q(a), q(b)$
Standing rule agenda	$p(a), p(b), q(a), q(b)$	$p(a), p(b), q(a), q(b)$

The standing tag is ordinary data interpreted by an explicit persistent, self-reinstalling dispatcher, not an additional native grammar. Its support stabilizes while work can remain. The checked source control proves eventual agreement for this instance; the general fairness theorem concerns arbitrary persistent positive rule systems.

Dependent evidence. A family can use a fact reading only through a specified fibre interpretation. A section additionally needs compatibility: equal observed facts must yield equal transported dependent values. Dependent quotient elimination constructs exactly these compatible sections, with both inverse laws and uniqueness. Finite-batch recovery is computed from actual first occurrences; it is not an assumed operation selecting a proof from mere provability.

The reachability example has destination-indexed result types $\mathrm{Fin}(d+1)$ and a compatible destination consumer. Route-dependent terms and fibres recording particular premise occurrences cannot use the same fact-only quotient. Actual execution actions also construct material members of the dependent continuation family, with inverse decoding; an empty unrequested space has an empty continuation fibre.

Two paths to the same fact retain their common query origin. A concrete two-outcome probability model makes the two derived indicators equal and nonconstant, refuting their independence. Conversely, disjoint origin names alone would not prove probabilistic independence. Counting proof paths, distinct facts and origin support are separate readings.

Generated variables and evidence levels. The strict reflective template interpreter rejects an uncovered output variable. The rule-scoped MM2 interpreter admits output-local variables, including a variable subsequently used by a generated rule's input. The retained native provider implements the latter behaviour. A blanket statement that "MM2 rejects unbound generated variables" is therefore false; the language and scope profile must be named. Captured payload syntax remains distinct from new rule-local binders.

The entry point `Mettapedia.GSLT.ProgrammableSpaces` and its qualification target collect the kernel results and declaration dependencies. Generic material, inference and descent constructions are checked without `Classical.choice`. Named concrete source comparisons inherit existing host dependencies, including native UTF-8 library proofs, arithmetic termination proofs in the existing wire codec, and the standard counted-production library; the qualification manifest records each affected declaration. Compiler runtime counterparts and runtime replay receive no kernel-theorem credit.

Native fixtures in `MettaKernel/Examples/ProgrammableSpaces` replay evaluation, generated transactions, scheduling and suspension against independent expectations. They qualify those executions, not compiled C in general. Native `set:` remains open: identification requires a declared reading and proved consumer compatibility.